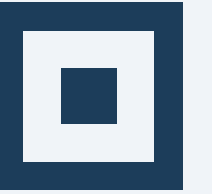


MCP: de copiar y pegar a operar archivos

Hernández Alonso Diego Raymundo · 2022361124 · 7CV4
Desarrollo de Aplicaciones Móviles Nativas · ESCOM IPN
Especificación MCP consultada: 2026-07-28



El problema: un LLM está aislado



Recibe texto y regresa texto

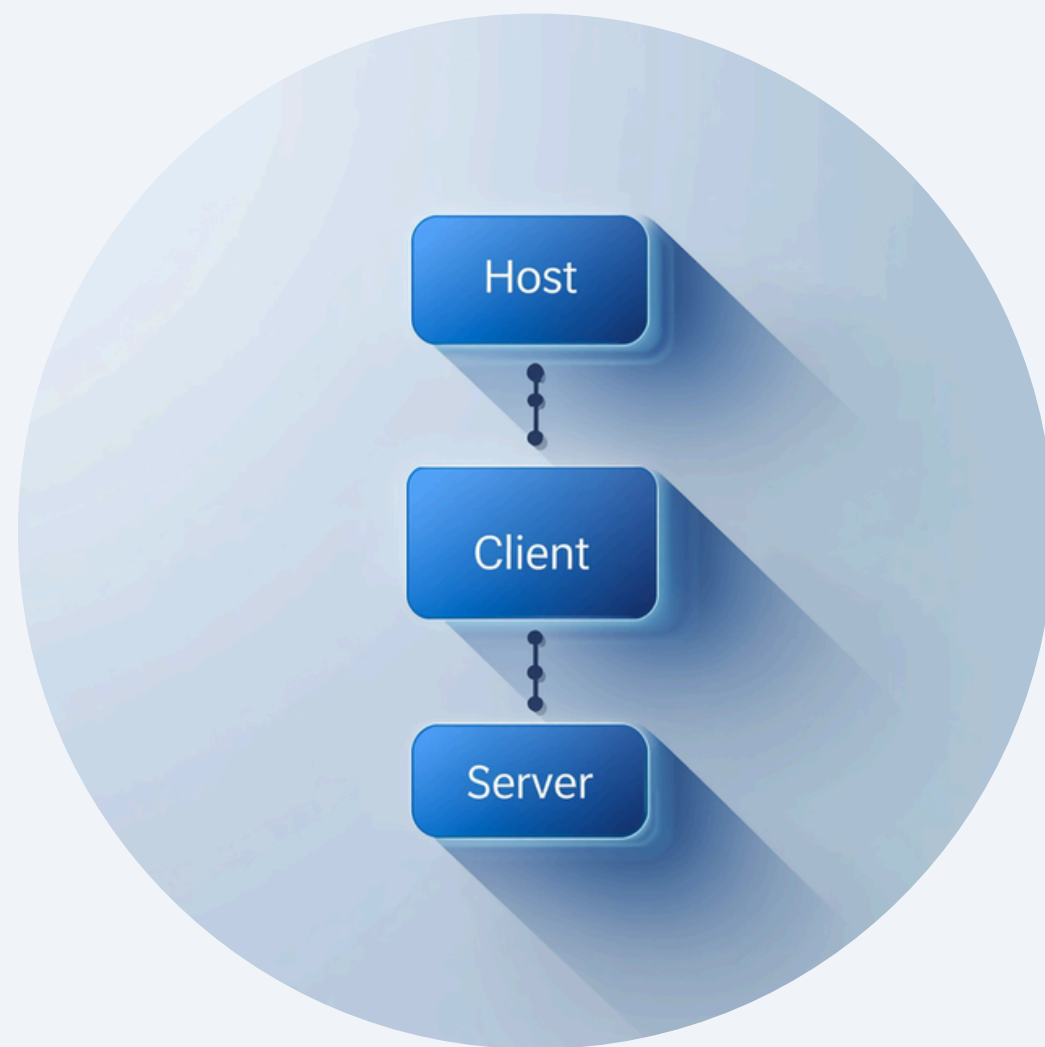
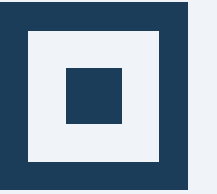
Arquitectura: corre en un servidor remoto, sin canal a mi disco ni llamadas al sistema operativo.

Seguridad: aislamiento, consentimiento del usuario e inyección de instrucciones.

Antes yo era el puente: copiar y pegar.

Los modelos que razonan tampoco tocan archivos: su razonamiento es texto (viene de entrenamiento por refuerzo y más cómputo al responder, no solo del tamaño).

La solución: host, cliente y servidor



Así quedó en mi compu

Host: Claude Desktop, la app que uso y la que pide permiso.

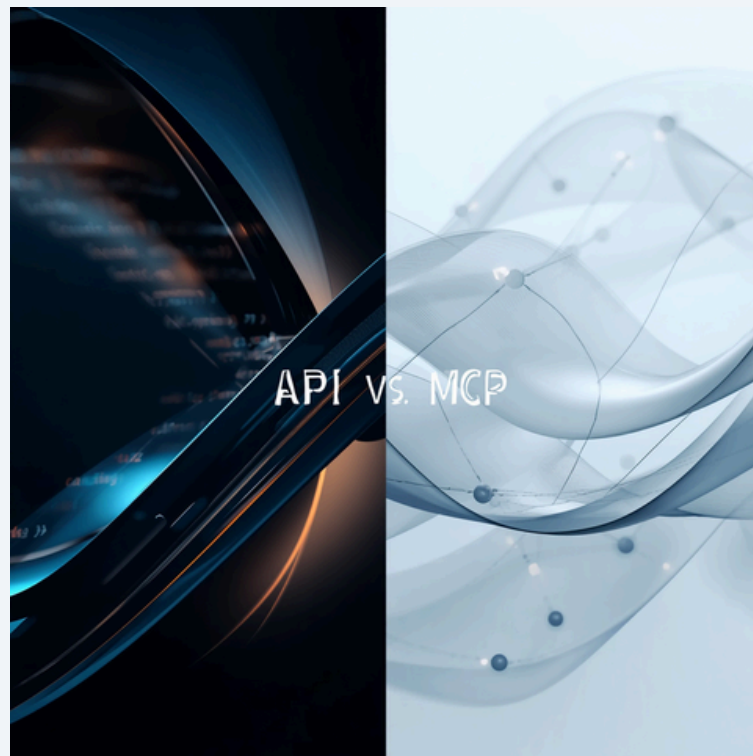
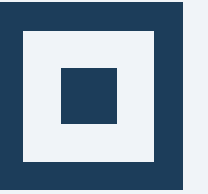
Cliente: el conector que el host crea para cada servidor (1 a 1).

Servidor: server-filesystem, un proceso de Node.js local.

El modelo solo propone qué herramienta usar; nunca toca el disco.

Modelo → Host → JSON-RPC 2.0 → Servidor MCP → Disco

API vs MCP: MCP no reemplaza a las APIs, las envuelve



API

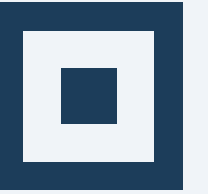
Decide el programador, al escribir el código
Se descubre leyendo documentación
Acoplamiento alto
Cada API con su formato
El permiso no es parte del contrato
 $N \times M$ integraciones



MCP

Decide el modelo, al momento, según lo que pide el usuario
Se descubre con tools/list
Acoplamiento bajo
Siempre JSON-RPC 2.0
El host debe pedir consentimiento
 $N + M$ piezas · protocolo abierto

Primitivas y Transportes en MCP



Qué ofrece cada lado y cómo viaja

Servidor: tools (acciones que decide el modelo), resources (datos por URI que adjunta la app) y prompts (plantillas que elige el usuario).

Cliente: elicitation (pedirle datos al usuario). Roots y sampling quedaron obsoletos en 2026-07-28.

stdio: servidor local como proceso hijo (el que uso).

Streamable HTTP: servidor remoto por POST, con OAuth 2.1 y ya sin sesiones.

[illegible]

No es parte del protocolo: es un servidor de referencia más

14 herramientas: 10 de lectura y 4 de escritura (write, edit, create directory, move) que requieren aprobación.

No tiene herramienta para borrar, y `search_files` busca por nombre, no por contenido.

Solo entra a C:\dev\Tarea1HernandezAlonso\sandbox.

En cada llamada normaliza la ruta (resuelve ..), revisa que empiece con la carpeta permitida y resuelve symlinks.

Demo en vivo: Operaciones de archivos

Las 5 operaciones

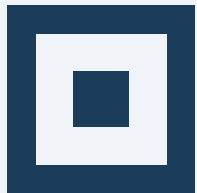
Listar (`list_directory`), leer `notas.md`, crear `evidencia.md` (`write_file`), modificar `pendientes.txt` (`edit_file`) y buscar `**/*.csv` (`search_files`).

Prueba del límite

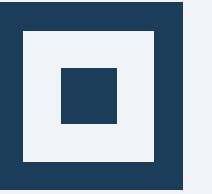
Primero `list_allowed_directories` (solo el sandbox). Luego `read_text_file` sobre `README.md`, que está una carpeta arriba.

Access denied

"path outside allowed directories". El modelo sí lo intentó; el que lo bloqueó fue el servidor. Con `sandbox\..\README.md` pasó lo mismo.



El problema que me encontré



La especificación cambió y el servidor no

Error de Schema

Claude Desktop solo acepta JSON Schema 2020-12 (especificación 2026-07-28) y el servidor manda draft-07. Probé de la versión 2025.3.28 a la 2026.8.31: todas igual.

Mi solución

Hice un adaptador en Node.js entre Claude Desktop y el servidor. Solo le quita la etiqueta \$schema a tools/list; el bloqueo de rutas sigue igual.

Servidor propio

Mi servidor en Python (SDK mcp 2.2.0) no necesitó el adaptador: 2 herramientas, 1 recurso y 1 prompt.

Seguridad y conclusión

- Riesgos: inyección de instrucciones por archivos, salir de la carpeta, sobrescribir sin querer
- Mitigaciones: Carpeta limitada, aprobación humana, versión fija, todo en Git
- El límite real está en el código del servidor, no en la buena voluntad del modelo.
- MCP redefine quién integra: el servidor se describe, el modelo decide y el usuario aprueba.